

Imposter Game Master Build Brief (v3)

Complete game design, architecture, and risk plan for AI-agent-assisted build

September 2026

- [1. What This Document Is](#)
- [2. Roles & Permissions](#)
 - [Host](#)
 - [Player](#)
 - [Audience \(people watching the host's screen who are not playing\)](#)
- [3. Config Rules \(Locked\)](#)
- [4. Full Game Flow](#)
 - [4.1 Host side, step by step](#)
 - [4.2 Player side, step by step](#)
 - [4.3 Round loop internals \(technical\)](#)
- [5. Scoring System](#)
 - [5.1 Base score for a correct vote \(same for 1- or 2-imposter games\)](#)
 - [5.2 Imposter scoring — exact definitions \(locked, corrected\)](#)
 - [5.3 Two-imposter games — each imposter scored independently](#)
 - [5.4 Confidence bet \(rounds 2-3 only\)](#)
 - [5.5 Winner & ties](#)
- [6. Visibility Matrix — What Each Role Can See, By Game Phase](#)
- [7. Locked Rulings on Every Edge Case](#)
- [8. Pre-Build Risk Plan — Loopholes to Resolve *Before* Writing Code](#)
- [9. Post-Deploy Risk Plan — What Only Shows Up in Production](#)
- [10. Scaling Plan — From a Few Rooms to 200+ Concurrent Players](#)
 - [10.1 What a single backend instance can actually handle](#)
 - [10.2 When to actually add horizontal scaling](#)
 - [10.3 Data-layer scaling](#)
 - [10.4 Never do this](#)
- [11. Final Tech Stack](#)
- [12. Build Checklist Summary](#)

1. What This Document Is

This is the single source of truth for building the Imposter Game — a web-based, real-time social deduction game for 3-12 players per room, with support for many rooms running at once. It is written to be handed to a build agent in one prompt: it covers the game rules, the visibility rules between host/player/audience, the full technical architecture, every edge case that has been explicitly ruled on, and the risk/scaling plan for both before and after launch.

Build this exactly as specified below. Where a rule is marked “locked,” it is a decision, not a suggestion — do not reinterpret it.

2. Roles & Permissions

Host

- Creates and configures the room; owns the QR code and join link.
- Sees: live lobby list, current round number, whose turn it is, the shared clue chat, a “voting in progress” status, and timers.
- **Never sees, until the game fully ends:** any score, any vote tally, who voted for whom, any confidence value, or a partial leaderboard.
- Can: force-start with fewer players than configured (with a warning), end the game early, and start a brand-new game (always a fresh room, never a reused one).

Player

- Joins exactly one room, via that room’s QR/link only.
- Sees: their own private identity (secret word, or “you are the imposter”), the lobby list, their own turn prompts, the shared clue chat, the voting screen (self excluded from vote options), and — in rounds 2-3 only — a confidence slider.
- **Never sees, until the final leaderboard:** their own running score, anyone’s identity but their own, or anyone’s votes.

Audience (people watching the host’s screen who are not playing)

- Sees only what the host’s **public display** shows: round number, whose turn it is, the clue chat, and “voting in progress.” Identical restriction to the host — no live scores, ever, during play. This is what stops a spectator from calling out the imposter mid-game.

3. Config Rules (Locked)

Players	Valid imposter count(s)	Notes
3-5	1	Below 3 players there’s no meaningful deduction — disallow entirely
6-9	1 (2 optional, with an on-screen balance warning)	
10-12	2	

- Rounds: host chooses 1, 2, or 3.
- Genre: cosmetic only — selects which word/topic pool is used.
- The confidence-bet mechanic (§5.4) is active **from round 2 onward only**. A 1-round game never uses it, regardless of genre or player count.

4. Full Game Flow

4.1 Host side, step by step

1. `CREATE_ROOM` — configure players, imposters (auto-limited per §3), rounds, genre.
2. Server issues a unique room code, QR, and link. Host enters `LOBBY_OPEN`.
3. Host watches the join count live (e.g. “6 / 8 joined”).
4. “Start” is enabled once the configured count has joined, or the host force-starts.
5. Start → server moves the room to `ROLE_REVEAL`, then `ROUND_1_CLUES`.
6. Per round: host’s screen shows only current turn/clue feed, then “voting in progress,” then `ROUND_N_COMPLETE` (no results) → next round’s lobby check → repeat.
7. After the final round: `GAME_END` → full leaderboard revealed to host, optionally cast to the public display.
8. “New Game” → always issues a **brand-new room code**; the old room becomes `ARCHIVED` (read-only, still viewable, never reused).

4.2 Player side, step by step

1. Scan QR → resolves to `{roomCode}` → `JOIN_ROOM` with a chosen name.
2. Server issues a session token bound to `{roomCode, playerId}`, stored client-side.

- 3. LOBBY_WAITING until the host starts.
- 4. ROLE_REVEAL (private) — this screen is **re-accessible at any time** during the game via a small persistent “view my role” control, not a one-time popup.
- 5. Per round:
 - CLUE_PHASE : on your turn, a 30-second server-driven timer; submit one clue into the shared chat.
 - VOTE_PHASE : 45-second timer; pick one other player (never yourself); rounds 2-3 also submit a confidence level (50-100%) at the same time as the vote.
 - ROUND_WAIT : no result information shown.
- 6. After the final round: LEADERBOARD , identical view for every player.

4.3 Round loop internals (technical)

- Turn order for a round is one server-side shuffle of currently active players, generated once at round start and stored. The server marks each player “answered” as they go — a player can never be selected twice or skipped within the same round.
- All timers are server-authoritative: the server stores an absolute deadline (epoch milliseconds) and clients only render a countdown to it. A missed deadline auto-scores 0 and the turn advances automatically — the game can never hang waiting on a client.

5. Scoring System

5.1 Base score for a correct vote (same for 1- or 2-imposter games)

Round	Correct-vote score
1	+1
2	+2
3	+3

A wrong vote or no vote scores 0, no exceptions.

5.2 Imposter scoring — exact definitions (locked, corrected)

- **N = number of active crew players eligible to vote that round** (includes anyone who timed out — a timeout is still a crew member who failed to catch the imposter).
- **C = number of those crew players whose submitted vote specifically names that imposter.**

Catch rate	Imposter’s score for the round
C = N	0
0 < C < N	half of that round’s max escape bonus, rounded down
C = 0	full max escape bonus

Max escape bonus by round: Round 1 = 2, Round 2 = 3, Round 3 = 4.

Why N must include timeouts: if timeouts were excluded from N, the imposter’s catch-rate percentage would shift just because fewer people bothered to vote — that’s a scoring loophole, not a real signal of how well the imposter blended in. Locking N to “everyone eligible” removes that loophole entirely.

5.3 Two-imposter games — each imposter scored independently

With imposters A and B, a vote for A never affects B’s score and vice versa. Votes cast against an innocent crew member count toward nobody’s catch rate.

Worked example — 10 players, 2 imposters, 8 crew:

5 crew vote for Imposter A
2 crew vote for Imposter B
1 crew times out (counts in N for both, contributes to neither's C)

Imposter A: C = 5, N = 8
Imposter B: C = 2, N = 8

It is entirely possible for A to be fully caught (score 0) while B escapes clean (full bonus) in the same round — this is expected behavior, not a bug.

5.4 Confidence bet (rounds 2-3 only)

Submitted **at the same time as the vote**, before the outcome is known — this is what prevents everyone from defaulting to “100% sure” for a free bonus:

Confidence	If correct	If wrong
90–100%	+2	−2
70–89%	+1	−1
50–69%	0	0

5.5 Winner & ties

- Winner = highest total score across all rounds.
- Tie-break order: (1) most correct votes overall → (2) most confidence points earned → 3. declared a shared win if still tied.
- There is **no plurality “most-voted-out” mechanic** in this game. Every vote is judged individually against ground truth, never tallied against other players’ votes — so classic “vote ties” and “confidence ties” simply don’t exist as a category here.

6. Visibility Matrix — What Each Role Can See, By Game Phase

Phase	Host sees	Player sees	Audience sees
Lobby	Join count, names	Names of others joined	Nothing (pre-game)
Role reveal	Nothing role-specific	Own identity only	Nothing
Clue phase	Whose turn, clue chat, timer	Own turn prompt (if theirs), clue chat, timer	Whose turn, clue chat, timer
Voting phase	“Voting in progress” only	Own voting screen only	“Voting in progress” only
Between rounds	Nothing about results	Nothing about results	Nothing about results
Game end	Full leaderboard	Full leaderboard (same view)	Host’s choice to cast leaderboard or not

This table is the enforcement spec for the backend: **any payload containing scores, vote counts, or identities must be filtered per-role at the server before it is ever emitted** — never filtered client-side, since a client-side-only filter can be bypassed by anyone opening dev tools.

7. Locked Rulings on Every Edge Case

Scenario	Ruling
Player disconnects during their clue turn	~20–30s grace period to reconnect and still submit within the original deadline; past deadline = auto 0, turn advances
Player disconnects during voting	Same grace period; past deadline = counted as no vote (0)
Host disconnects during an active turn	Game does not pause. All state/timers live server-side, not on the host’s device. Only the public display goes blank until the host reconnects
Player refreshes before role reveal	Session token already bound to their seat — resumes at lobby, no data lost
Player refreshes after role reveal	Resumes at the current game phase; role is always re-viewable via a persistent control
Player refreshes during their own turn	Reconnects into the same turn; countdown continues from the original server deadline
Player refreshes after submitting a clue	Shows “waiting for others”; resubmission blocked server-side (idempotent per round+player)
Player opens another room’s QR/link while active elsewhere	Blocked with a prompt to leave the current game first — one active room per session
Two browser tabs, same player	Most recently active tab is authoritative; other tabs go read-only (“opened elsewhere”)
Host opens the same game in two tabs	Same pattern; all host actions are idempotent server-side regardless
2 imposters, one disconnects permanently	Marked inactive; their catch rate locks at whatever was recorded; the other imposter is unaffected
Player leaves permanently after round 1	Marked inactive; excluded from future “did everyone answer” checks; keeps points already earned
All players time out their votes	Valid outcome: imposter(s) get the full escape bonus, crew all score 0 that round
“Vote ties” / “confidence ties”	Don’t apply — see §5.5, nothing is tallied against other players’ choices
1-round game with confidence	Mechanic is off entirely; round 1 never has it regardless of total rounds
Can a player see their own score mid-game	No — hidden until the final leaderboard, same principle as hiding it from host/audience

8. Pre-Build Risk Plan — Loopholes to Resolve *Before* Writing Code

Risk	Why it matters	Resolution to build in from day one
Turn order repeats or skips a player	Breaks the core fairness guarantee of the game	Generate the shuffle once per round server-side; track “answered” as a set the server owns, never derived from the client
Client-side rule bypass (self-vote, double submit, voting after deadline)	A technical player can call the API directly and skip the UI entirely	Every rule must be re-checked server-side, with zero trust in what the UI allowed
Rooms bleeding into each other	The single most important guarantee of this app	Rooms as first-class namespaces; a message emitted in Room A must be structurally incapable of reaching Room B’s sockets — test with 3+ concurrent rooms before anything else
Room code collisions	Two hosts could get the same code under load	Generate against a uniqueness check on active rooms before issuing, never a bare random draw
Race conditions on turn/round advancement	Two near-simultaneous submissions racing to “complete” a turn	Treat round/turn advancement as one atomic operation per room, never something two events can trigger independently
Timer drift on the client	Backgrounded mobile tabs throttle <code>setInterval</code>	Never trust client elapsed time; always recompute the countdown from <code>serverDeadline - now()</code>
Scoring loophole from excluding timeouts	Imposter’s apparent catch rate would shift just because fewer people voted	Locked in §5.2 — timeouts count in N, never in C
No plan for the 2-imposter split-vote case	Ambiguous scoring is worse than no scoring	Locked in §5.3 — each imposter scored fully independently

Recommended order to build these in: room/session isolation → lobby/QR joining → private role assignment → round state machine → server-controlled clue turns/timers → voting → server-side scoring → round transitions → final leaderboard → refresh/reconnect/multi-tab handling → concurrent-room testing → load testing → deploy.

The single most important test before considering this “working”:

Host A → Room A → 8 players Host B → Room B → 8 players Host C → Room C → 12 players
Then simultaneously: A refreshes, B loses network, a player in A refreshes, a player in B opens a second tab, C starts Round 2.
Expected: A sees only A. B sees only B. C sees only C. No player crosses rooms. No duplicate turns. No duplicate votes. No timer resets. No role leakage. No score leakage.

If this scenario doesn’t hold, the architecture is wrong regardless of whether the UI looks fine in a single-room manual test.

9. Post-Deploy Risk Plan — What Only Shows Up in Production

Risk	Why it only shows up after launch	Mitigation
Server restart wipes in-memory game state mid-game	A redeploy or crash during a live game shouldn’t erase it	Persist active-game state to Redis (or equivalent) so rooms rehydrate on restart instead of vanishing
Backgrounded mobile tabs miss “your turn”	Real phones get locked/backgrounded constantly; a local dev browser tab never does	Server timers already protect scoring correctness (§4.3); add a sound/vibration/notification cue for “your turn”
Stale client bundle after a deploy	A player who joined before a deploy is running old JS against a new server contract	Version the client build; force a refresh prompt on any protocol mismatch
Zombie/abandoned rooms	Hosts who close a tab and never return leave rooms running forever, wasting memory	A cleanup job archives rooms inactive beyond a timeout (e.g. 15+ minutes of no host activity)
Load spikes (a whole classroom/event starting at once)	Steady-state testing doesn’t reveal burst behavior	Load-test concurrent room creation specifically, not just steady traffic, before any real event

10. Scaling Plan — From a Few Rooms to 200+ Concurrent Players

Important framing: a single room is capped at 12 players by design, so “200 players” in practice means many rooms running at once (roughly 17–65 rooms depending on room size), not one giant room. That distinction changes the engineering answer significantly.

10.1 What a single backend instance can actually handle

A single well-configured Node.js + Socket.IO process can comfortably hold several thousand concurrent WebSocket connections on modest hardware. **200 concurrent players across many small rooms is well within what one backend instance can serve — this does not need to be “solved” with heavy infrastructure on day one.** The real risks at this scale are not raw connection count, they are: - Memory used per room (keep each room’s state small and flat — player list, turn order, current deadline, votes-so-far — not large chat histories held forever in memory). - CPU spent on frequent small broadcasts (batch/limit chat and timer-tick broadcasts sensibly rather than emitting on every millisecond). - One slow room or client should never block another room’s event loop — keep all per-room logic non-blocking and avoid synchronous heavy work inside socket handlers.

10.2 When to actually add horizontal scaling

Add a second backend instance (and the coordination that requires) only once you’re consistently approaching a single instance’s real ceiling — not preemptively. When you do: - Use the **Redis adapter for Socket.IO** so events broadcast correctly across multiple backend instances (a client on instance 2 still receives events from a room whose host is connected to instance 1). - Store room state in Redis rather than only in a given instance’s memory, so any instance can serve any room and a reconnect doesn’t have to land back on the exact same instance. - Put a load balancer in front with either sticky sessions or, better, full statelessness at the instance level via the Redis-backed room state above.

10.3 Data-layer scaling

- Supabase/Postgres is only ever written to for **completed** rounds/games, not live per-second state — so its load stays low and predictable even as concurrent live rooms grow, since it never sees the high-frequency traffic (that all stays in memory/Redis).
- Watch Supabase’s connection-pool limits as room-completion writes scale up; use its built-in connection pooler (PgBouncer-based) rather than opening a new raw connection per request.

10.4 Never do this

- Never store game state as a single global object (`global.gameState = ...`). Always key it as `rooms.get(roomId)` , with every socket event authenticated against that specific room. This is the actual mechanism that prevents cross-room leakage at any scale — it matters more at 200 concurrent players than at 12, because more rooms means more chances for a shared-global bug to surface.
- Never assume “add more servers” fixes a correctness bug. If Room A and Room B ever leak into each other on one instance, adding a second instance does not fix it — it just makes the bug intermittent and harder to reproduce. Fix isolation first (§8), scale second.

11. Final Tech Stack

Layer	Choice	Why
Frontend (host + player UI)	React + Tailwind, built as a PWA, deployed on Vercel	Fast to build distinct host/player/audience views; PWA lets players add-to-home-screen
Real-time layer	Socket.IO (rooms/namespaces)	Native support for exactly the isolation boundary this game needs
Backend	Node.js + Express + Socket.IO , deployed on Render, Railway, or Fly.io (not Vercel)	Needs a persistent, always-running process to hold server-authoritative timers and in-memory room state; Vercel’s WebSocket support (public beta since June 2026) caps connection duration and doesn’t guarantee instance affinity on reconnect, which conflicts with this game’s 30-45s server-timed turns
Database	Supabase (PostgreSQL)	Stores completed games, final leaderboards, archived rooms, optional host accounts — never live per-second state
Hot game state	Backend memory initially → Redis once horizontal scaling is actually needed (§10.2)	Keeps day-one architecture simple without requiring a rewrite later — build the state layer behind a <code>GameStateStore</code> interface from the start so swapping memory for Redis is a config change, not a rewrite
Session identity	Short-lived signed token (JWT), issued at join, scoped to <code>{roomCode, playerId}</code> , stored client-side	This is the mechanism behind refresh-without-losing-your-seat and one-active-room-per-session enforcement
Host accounts (optional)	Supabase Auth (magic link/Google), only if hosts need to see past games across devices	Not required for v1 if hosts are fine with session-only identity like players

Deployment split (important): frontend on Vercel, backend on Render/Railway/Fly.io, Supabase stays as-is since it’s reachable over HTTP from anywhere. Do not attempt to run the Socket.IO backend on Vercel for this project.

12. Build Checklist Summary

- Room/session isolation (namespaces + signed session tokens).
- Lobby + QR joining flow.
- Private per-socket role assignment (server never broadcasts identity to the wrong client).

- 4. Round state machine: CREATE → LOBBY → ROLE_REVEAL → ROUND_CLUES → VOTING → ROUND_WAIT → NEXT_ROUND / GAME_END → ARCHIVED .
- 5. Server-controlled clue turns and timers (deadline-based, never client-trusted).
- 6. Voting, with server-side self-vote and duplicate-submission rejection.
- 7. Scoring exactly per §5, including the corrected N/C definition.
- 8. Round transitions and final leaderboard reveal.
- 9. Refresh/reconnect/multi-tab handling per §7.
- 10. Concurrent-room testing (§8’s three-host scenario) before anything else is called “done.”
- 11. Load testing toward the scaling plan in §10.
- 12. Production deployment per §11’s split.

Every rule in this document is intended to be implemented as written — where a table gives an exact number or formula, use that number, not an approximation.